

EXPERIMENT 8

8a) SIMULATION AND ANALYSIS OF BPSK MODULATION USING CONSTELLATION DIAGRAMS IN MATLAB

OBJECTIVE:

- i) To generate BPSK symbols from a binary data sequence using phase-shift keying principles.
- ii) To Plot and analyze the BPSK signal constellation in the In-Phase (I) and Quadrature (Q) plane.
- iii) To investigate the effect of noise on constellation points and evaluate the resulting degradation in symbol detection performance.

PROGRAM:

```
clc;
```

```
clear;
```

```
close all;
```

```
%% Objective 1: Generate BPSK Symbols
```

```
N = 2000;           % Number of bits
```

```
data = randi([0 1], N, 1); % Random binary sequence
```

```
% BPSK Mapping
```

```
bpsk_symbols = 2*data - 1;
```

```
%% Objective 2: Plot BPSK Signal Constellation
```

```
figure;
```

```
subplot(1,2,1);
```

```
scatter(real(bpsk_symbols), imag(bpsk_symbols), 25, 'filled');
```

```
hold on;
```

```
xline(0,'k','LineWidth',1.5);
yline(0,'k','LineWidth',1.5);
grid on;
axis equal;
xlim([-1.5 1.5]);
ylim([-1 1]);
xlabel('In-Phase (I)');
ylabel('Quadrature (Q)');
title('Ideal BPSK Constellation');
```

%% Objective 3: Add Noise and Evaluate Performance

```
SNR = 10; % Signal-to-Noise Ratio (dB)
```

```
rx_signal = awgn(bpsk_symbols, SNR, 'measured');
subplot(1,2,2);
scatter(real(rx_signal), imag(rx_signal), 12, 'filled');
hold on;
xline(0,'k','LineWidth',1.5);
yline(0,'k','LineWidth',1.5);
grid on;
axis equal;
xlim([-2.5 2.5]);
ylim([-2 2]);
xlabel('In-Phase (I)');
ylabel('Quadrature (Q)');
title(['Noisy BPSK Constellation (SNR = ', num2str(SNR), ' dB)']);
```

%% Symbol Detection

```
detected_bits = real(rx_signal) > 0;
```

```
%% Calculate Errors
```

```
num_errors = sum(data ~= detected_bits);
```

```
BER = num_errors/N;
```

```
%% Display Results
```

```
fprintf('Number of Transmitted Bits : %d\n', N);
```

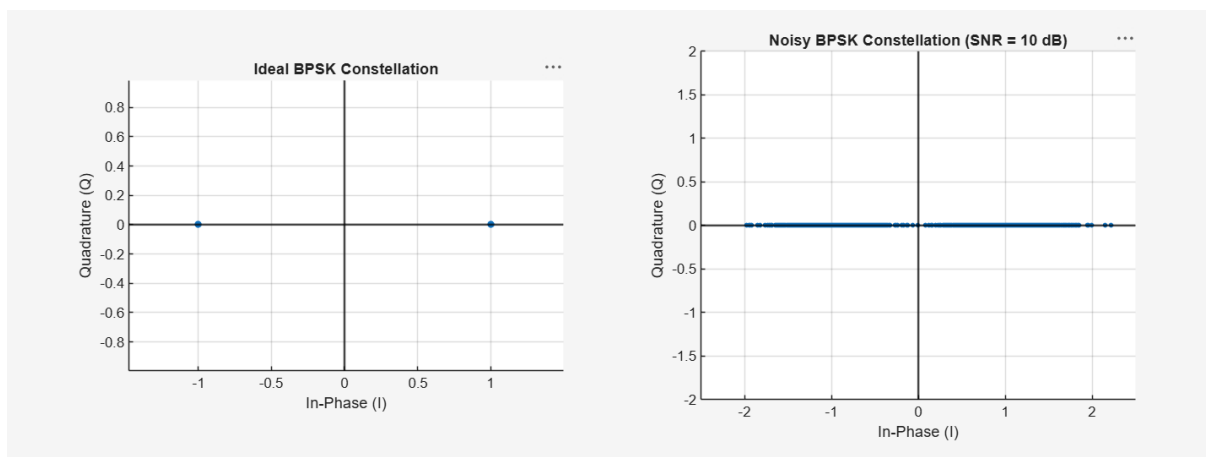
```
fprintf('Number of Received Bits : %d\n', N);
```

```
fprintf('Signal-to-Noise Ratio : %d dB\n', SNR);
```

```
fprintf('Number of Bit Errors : %d\n', num_errors);
```

```
fprintf('Bit Error Rate (BER) : %.6f\n', BER);
```

OUTPUT:



```
Number of Transmitted Bits : 2000
Number of Received Bits : 2000
Signal-to-Noise Ratio : 10 dB
Number of Bit Errors : 2
Bit Error Rate (BER) : 0.001000
>>
```

8b) SIMULATION AND ANALYSIS OF QPSK MODULATION USING CONSTELLATION DIAGRAMS IN MATLAB

OBJECTIVES:

- i) To generate QPSK symbols from a binary data sequence using quadrature phase-shift keying principles.
- ii) plot and analyze the QPSK signal constellation in the In-Phase (I) and Quadrature (Q) plane.
- iii) To investigate the effect of noise on constellation points and evaluate the resulting degradation in symbol detection performance.

PROGRAM:

```
clc;
```

```
clear;
```

```
close all;
```

```
%% Objective 1: Generate QPSK Symbols
```

```
N = 2000;           % Number of bits
```

```
data = randi([0 1], N, 1); % Random binary sequence
```

```
% Ensure even number of bits
```

```
if mod(N,2) ~= 0
```

```
    data = [data; 0];
```

```
    N = N + 1;
```

```
end
```

```
% Group bits into pairs
```

```
bit_pairs = reshape(data,2,[]);
```

```
% QPSK Mapping
```

```
qpsk_symbols = zeros(size(bit_pairs,1),1);
```

```
for k = 1:size(bit_pairs,1)
```

```
    if isequal(bit_pairs(k,:),[0 0])
```

```
        qpsk_symbols(k) = (1+1j)/sqrt(2);
```

```
    elseif isequal(bit_pairs(k,:),[0 1])
```

```
        qpsk_symbols(k) = (-1+1j)/sqrt(2);
```

```
    elseif isequal(bit_pairs(k,:),[1 1])
```

```
        qpsk_symbols(k) = (-1-1j)/sqrt(2);
```

```
    elseif isequal(bit_pairs(k,:),[1 0])
```

```
        qpsk_symbols(k) = (1-1j)/sqrt(2);
```

```
    end
```

```
end
```

```
%% Objective 2: Plot QPSK Constellation
```

```
figure;
```

```
subplot(1,2,1)
```

```
scatter(real(qpsk_symbols), imag(qpsk_symbols),25,'filled');
```

```
hold on;
```

```
xline(0,'k','LineWidth',1.5);
```

```
yline(0,'k','LineWidth',1.5);
```

```
grid on;
```

```
axis equal;
```

```
xlim([-1.5 1.5]);
```

```
ylim([-1.5 1.5]);
```

```
xlabel('In-Phase (I)');
```

```
ylabel('Quadrature (Q)');
```

```
title('Ideal QPSK Constellation');
```

```
%% Objective 3: Add Noise and Evaluate Performance
```

```
SNR = 12; % Signal-to-Noise Ratio (dB)
```

```
rx_signal = awgn(qpsk_symbols,SNR,'measured');
```

```
subplot(1,2,2)
```

```
scatter(real(rx_signal),imag(rx_signal),15,'filled');
```

```
hold on;
```

```
xline(0,'k','LineWidth',1.5);
```

```
yline(0,'k','LineWidth',1.5);
```

```
grid on;
```

```
axis equal;
```

```
xlim([-2 2]);
```

```
ylim([-2 2]);
```

```
xlabel('In-Phase (I)');
```

```
ylabel('Quadrature (Q)');
```

```
title(['Noisy QPSK Constellation (SNR = ',num2str(SNR),' dB)']);
```

```
%% Symbol Detection
```

```
detected_bits = zeros(length(rx_signal)*2,1);
```

```
for k = 1:length(rx_signal)
```

```
    I = real(rx_signal(k));
```

```
    Q = imag(rx_signal(k));
```

```

if I>0 && Q>0
    bits = [0 0];
elseif I<0 && Q>0
    bits = [0 1];
elseif I<0 && Q<0
    bits = [1 1];
else
    bits = [1 0];
end

detected_bits(2*k-1:2*k) = bits;

end

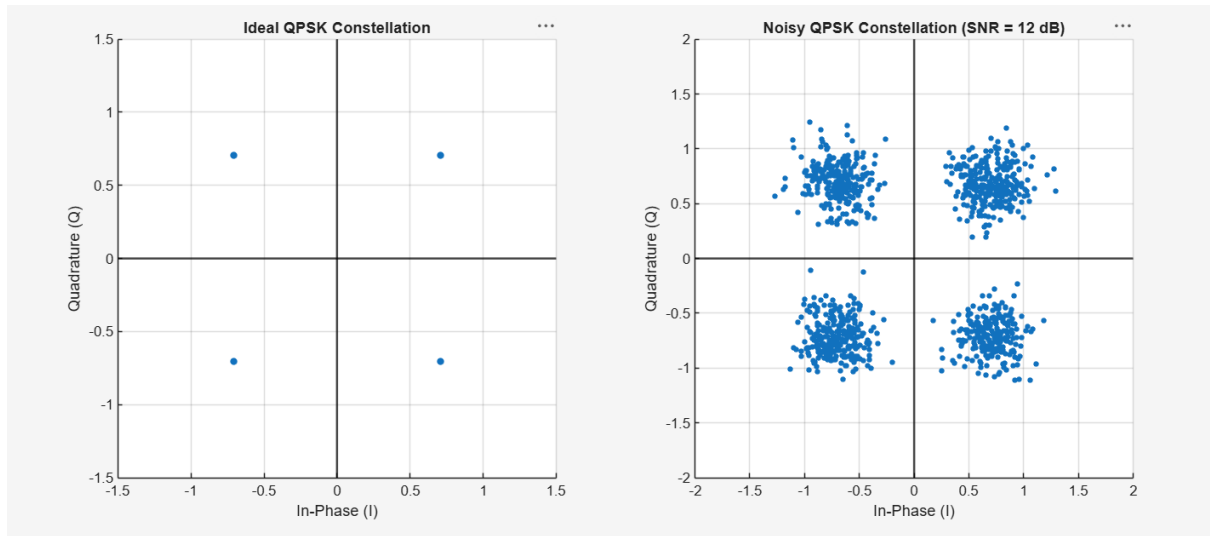
%% BER Calculation

num_errors = sum(data ~= detected_bits);
BER = num_errors/N;

fprintf('Number of Transmitted Bits : %d\n',N);
fprintf('Number of Received Bits   : %d\n',N);
fprintf('Signal-to-Noise Ratio      : %d dB\n',SNR);
fprintf('Number of Bit Errors         : %d\n',num_errors);
fprintf('Bit Error Rate (BER)          : %.6f\n',BER);

```

OUTPUT:



```
Number of Transmitted Bits : 2000
Number of Received Bits   : 2000
Signal-to-Noise Ratio     : 12 dB
Number of Bit Errors      : 0
Bit Error Rate (BER)      : 0.000000
>>
```